

Web Security - Session

**CSCI 4130: Introduction to Computer
Security**

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

不同身份验证或标识方法的 pros & cons

Web Authentication

- IP address

攻击者可以伪造源IP地址，无法完全信任

- The IP source address can be spoofed

多个用户可能共享同一公网IP（如公司或家庭网络），难以区分具体用户

- NAT-ing may cause several users to share the same IP address

同一用户可能在不同时间使用不同IP

- The same user could use different IP addresses

- HTTP

扩展性差

- Not very scalable and difficult to manage at the application level

- Certificate

服务端证书可验证服务器身份（如

（适用于TLS）

HTTPS防中间人攻击）

- Works (on the server-side) for TLS-based connections

大多数用户没有个人证书，且部署复杂

- Few users have “real” certificates or know how to use them

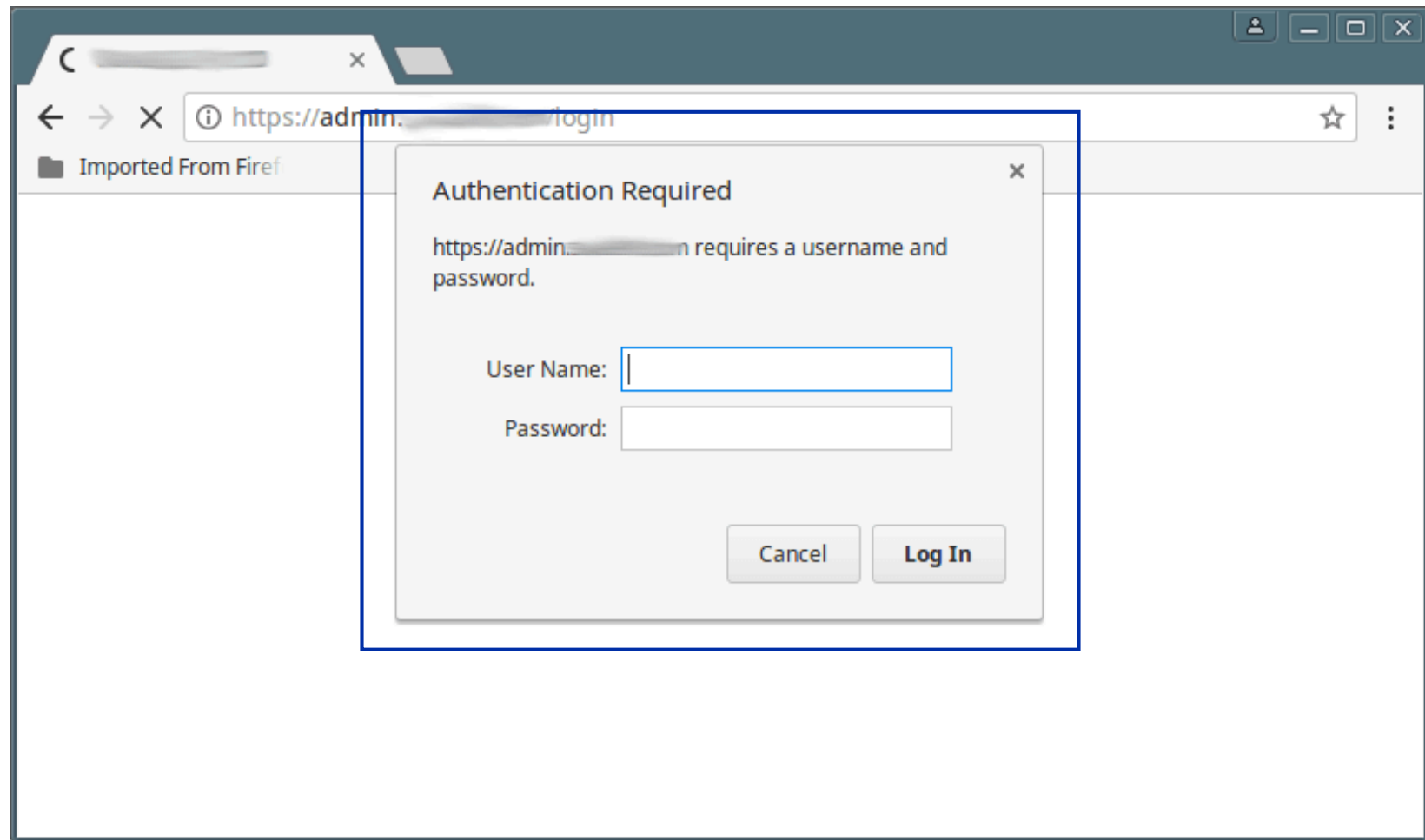
- Form

明文

- Form data might be sent in plaintext

可能会被窃听（因为发的是明文）

HTTP Authentication



The user is challenged for providing a username and password

HTTP Authentication (Cont.)

HTTP基本认证

挑战 - 响应

- It is a protocol based on simple challenge-response scheme
- The challenge is returned by the server as part of a 401 reply message
 - The message specifies the authentication schema to be used 该信息指定了要使用的身份验证模式
- An authentication request refers to a realm is sent by the client
 - 客户端发送的认证请求是针对服务器之前指定的某个 realm (安全区域) 的。
 - The realm specifies a group of resources on the server
- The client must include an Authorization header field with valid credentials
 - 客户端 (比如浏览器) 必须在HTTP请求里加一个 Authorization 头, 里面放正确的用户名密码 (或其他认证信息), 否则服务器会拒绝访问

核心流程

1. 服务器挑战 (401 Unauthorized)

- 当客户端访问受保护资源时，服务器返回 HTTP 401 状态码，并在响应头中声明认证方式（如 **Basic**）和 **realm**（资源分组标识，例如 "Admin Area"）。

```
http 复制
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Basic realm="Protected Area"
```

2. 客户端响应

- 客户端需弹出登录框，用户输入凭据（用户名/密码）。
- 凭据按 **username:password** 格式拼接，用 **Base64** 编码（注意：仅是编码，非加密！），通过 **Authorization** 请求头发送：

```
http 复制
GET /protected HTTP/1.1
Authorization: Basic dXNlcjpwYXNzd29yZA==
```

3. 服务器验证

- 服务器解码凭据，验证通过后返回资源，否则重复 401 流程。

1. realm 是什么？

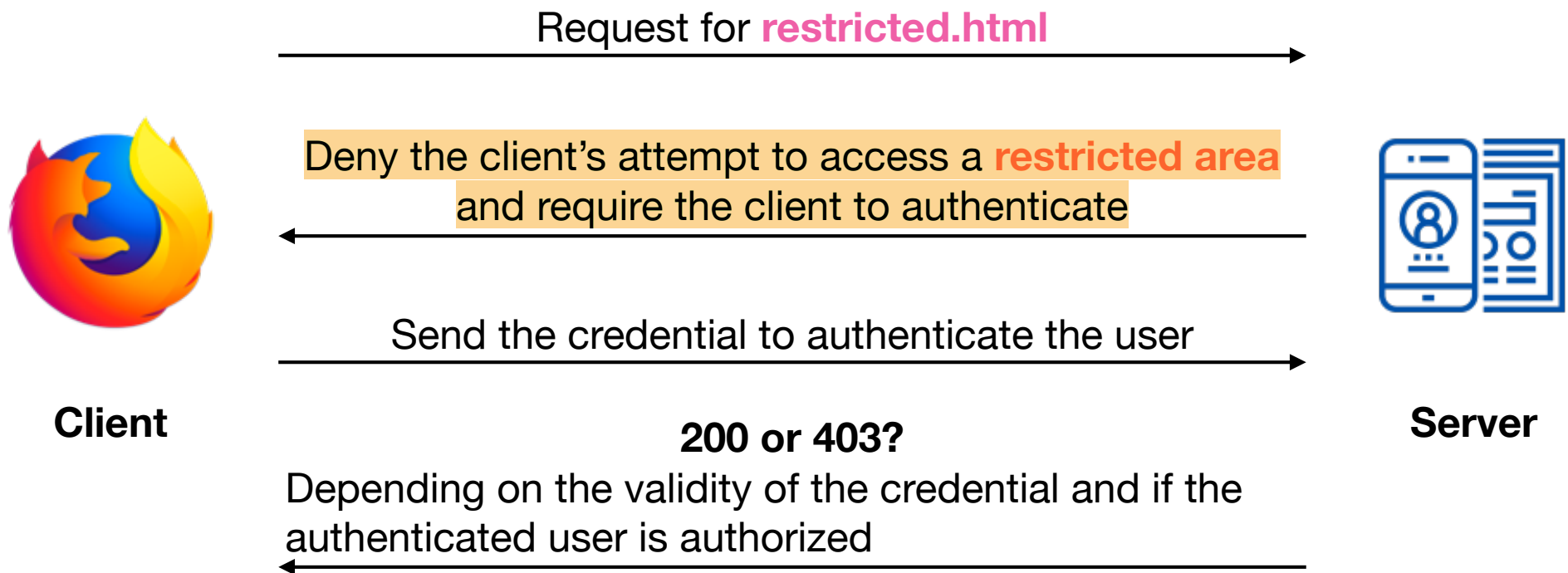
- 它是一个名称（比如 "Admin Panel" 或 "Financial Data"），告诉用户或客户端 当前要访问的资源属于哪个“安全区域”。
- 例如：
 - 访问公司网站时，**realm** 可能是 "Employee Portal"。
 - 访问银行系统时，**realm** 可能是 "Online Banking"。

2. 为什么需要 realm？

- 帮助用户区分不同的登录场景：
如果同一服务器有多个受保护区域（比如 **/admin** 和 **/finance**），**realm** 会告诉用户：“你现在要登录的是哪个部分？”
 - 比如：
 - 访问 **/admin** → 返回 **realm="Admin Dashboard"**
 - 访问 **/finance** → 返回 **realm="Accounting System"**
 - 这样用户知道自己在输入哪个系统的密码。
- 让客户端管理不同区域的凭据：
浏览器或客户端可以为不同的 **realm** 存储不同的用户名/密码，避免混淆。

HTTP Authentication (Cont.)

- From the web server's perspective, HTTP authentication enables basic **access control**



What is “401 Unauthorized”?

Version Status Status Message

HTTP/1.1 401 Unauthorized

Headers

...

Content-Type: text/html

WWW-Authenticate: Basic realm=“xxx”

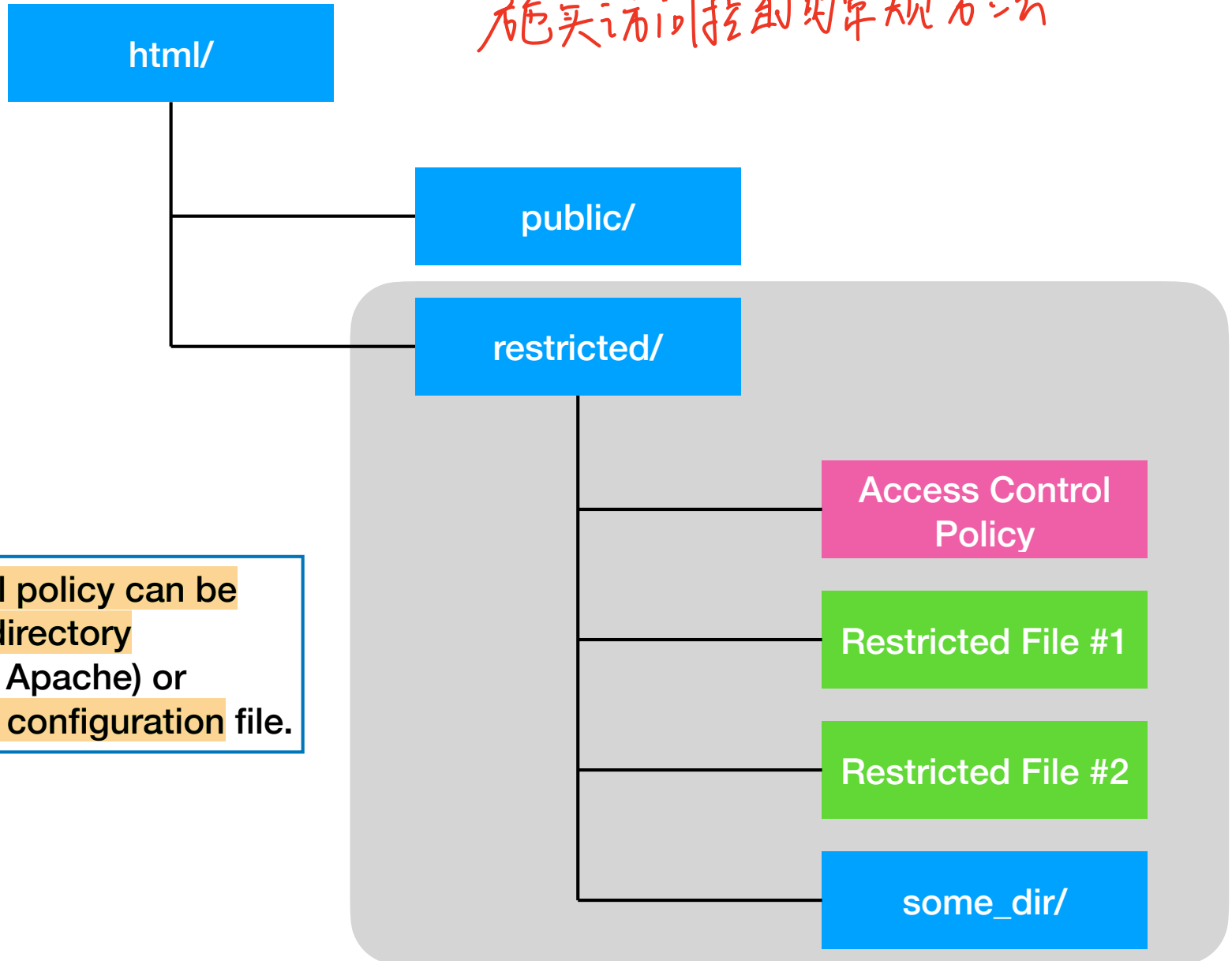
...

The “**WWW-Authenticate**” HTTP Header

1. **Basic / Digest:** It is about how the message is encoded.
2. **Realm:** It is about the name of the restricted area.

Normal Way To Enforce Access Control In A Web Server

实现访问控制的常规方法



The access control policy can be placed inside the directory (e.g., `.htaccess` for Apache) or defined in a global configuration file.

HTTP Is Stateless

HTTP 协议本身是无状态的，即服务器默认不会记住客户端的前一次请求。每次请求都是独立的，服务器不会自动关联同一用户的多次请求。

- The **HTTP server** is **not required** to retain information or status about each user for the duration of multiple requests.

用户访问页面A后跳转到页面B，服务器不会“记得”用户之前访问过A，除非通过额外手段实现。

- A **web application** **can** track state over multiple requests if it wants to.

- HTTP cookies 服务器通过 Set-Cookie 响应头向客户端（浏览器）发送一小段数据，客户端后续请求会自动通过 Cookie 头回传该数据

- Server side sessions 服务器端会话 服务器在内存或数据库中存储用户状态（如购物车数据），仅通过Cookie或URL传递一个会话ID来关联用户。

1. 用户登录 → 服务器生成 `session_id` 并存储到数据库 → 通过Cookie返回 `session_id`。
2. 用户下次请求携带 `session_id` → 服务器查询数据库恢复用户状态。

- Embedding information in the returned page 页面内嵌信息

- Hidden variables within web forms

- Modified URLs

实现状态跟踪的5种方法

User Sessions

会话

- A **session** is a sequence of HTTP request and response transactions associated to a user
- It is used to represent a **time-limited interaction** of a client with a server
- Session is a concept beyond the HTTP, thus needs to be implemented at the application level

Session 通常有有效期（如用户登录后30分钟无操作则自动过期）。
示例：电商网站若检测到用户长时间未活动，会要求重新登录。
同一个Session内的所有请求属于同一用户（通过Session ID关联）

HTTP协议本身不支持Session，需由开发者通过应用层代码实现



Client

HTTP Request #1
HTTP Response #1
HTTP Request #2
HTTP Response #2



Server

Basic Authentication

通过 HTML 表单, 向服务端提供凭据

- A form is used to send the credentials (over an TLS-protected channel) to a server-side application
- The application (服务器收到表单数据后)
 - Verifies the credentials 检查用户名和密码是否匹配数据库中的记录
 - 若验证通过, 生成一个 会话认证标识 (Session Authenticator)
 - Creates and sends back a session authenticator (typically a cookie, e.g., auth="Alice:Smith")
- The browser (浏览器会在后续的HTTP请求中自动携带认证标识
浏览器会自动在所有同域请求的 Cookie 请求头中加入这个标识, 无需用户手动操作)
 - Include the authenticator in the next requests
- Authentication is performed by checking this value

检验是否合法

检验是否过期

在 HTTP 无状态协议 中，帮助网站记住用户信息
或状态（如登录状态、购物车内容）。

HTTP Cookie

由 网站服务器生成 的一小段数据（通常为键值对，如 user_id=123），通过 HTTP 响应头（Set-Cookie）发送给用户的浏览器，并由浏览器 本地存储

- A **small** piece of data sent from/set by a website
- Stored on the **user's computer** by a web browser
- **Automatically sent** with a HTTP request by the browser
- Designed as a reliable mechanism to help web applications keep states at the client side
 - Logging in
 - Shopping cart
 - Last visited pages
 - ...
- Cookies are **accessible only by the website that sets them**

Cookie 只能被设置它的网站（域名）读取，其他网站无法偷看或篡改

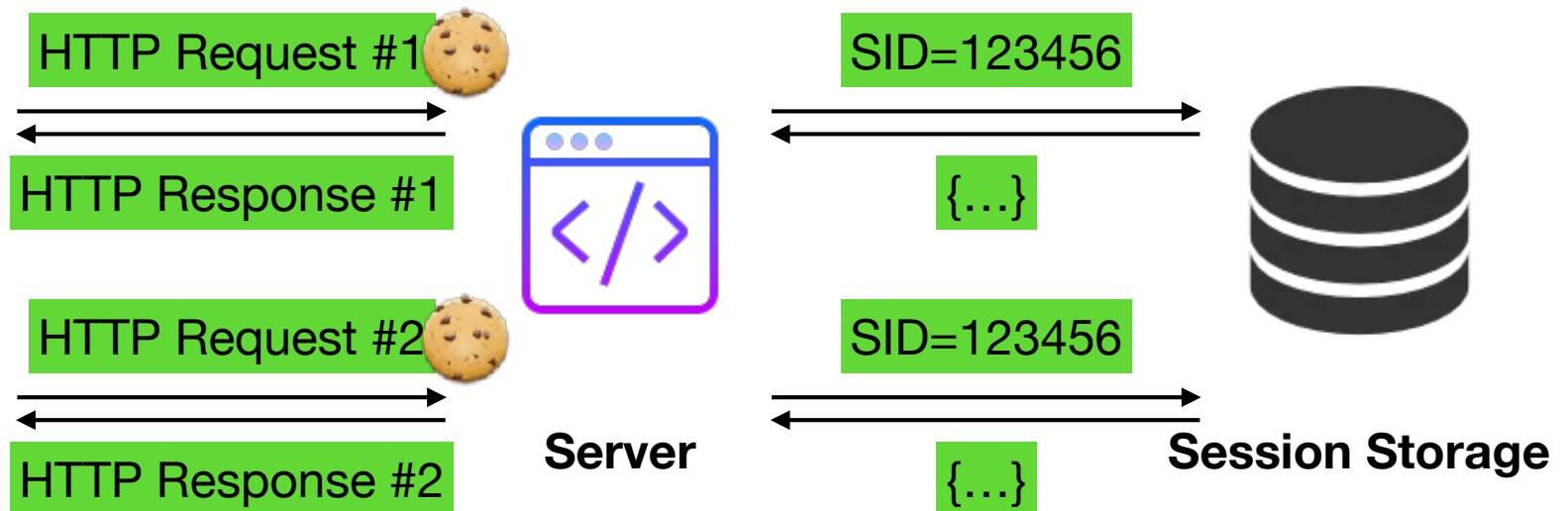
HTTP Cookie (Cont.)



Cookie Key	Cookie Value
...	...
SID	123456



Client



- A website can set multiple cookies for a client
- The cookies are stored by the web browser in a cookie jar
- The cookies are organized by their domain name
- Only cookies belonging to a domain A are sent with requests to domain A

HTTP Cookie: Under The Hood



Client



Server

GET index.html HTTP/1.1

HTTP/1.1 200 OK

Set-Cookie: SIDCC=AAiTGe; expires=Tue, 01-May-2018
14:43:54 GMT; path=/; domain=.example.com

POST login.html HTTP/1.1

Cookie: SIDCC=AAiTGe

HTTP/1.1 200 OK

Set-Cookie: SSID=A_siLF1Lx; expires=Tue, 01-May-2018
14:43:55 GMT; path=/; domain=.example.com

GET index.html HTTP/1.1

Cookie: SIDCC=AAiTGe; SSID=A_siLF1Lx

Cookie Attributes

Cookies set by docs.foo.com

HTTP/1.0 200 **OK**

Set-Cookie: LSID=DQAAAK...Eaem_vYg; Path=/accounts; Expires=Wed, 13 Jan 2021

22:23:01 GMT; Secure; HttpOnly

Set-Cookie: HSID=AYQEVn...DKrdst; Domain=.foo.com; Path=/; Expires=Wed, 13 Jan 2021

22:23:01 GMT; HttpOnly

Set-Cookie: SSID=Ap4P...GTEq; Domain=.foo.com; Path=/; Expires=Wed, 13 Jan 2021

22:23:01 GMT; Secure; HttpOnly

Set-Cookie: made_write_conn=1295214458; Path=/; Domain=.foo.com

Set-Cookie: reg_fb_gate=deleted; Expires=Thu, 01 Jan 1970 00:00:01 GMT; Path=/;

Domain=.foo.com; HttpOnly

...

- The **Domain** and **Path** attributes define the scope of the cookie.
 - If **Domain** is **not** specified, the cookie is set as a host-only cookie (exception: Internet Explorer).
不设 Domain, 设置为精确域名即可
 - If **Path** is specified, the cookie is only used when requesting pages contained in the corresponding path.
 - If **Path** is **not** specified, the path of the requested resource is used.



1. "If Domain is not specified, the cookie is set as a host-only cookie (exception: Internet Explorer)."

- 含义:

如果设置Cookie时不指定 `Domain` 属性, 则该Cookie默认成为 host-only cookie (仅限当前精确域名), 但Internet Explorer浏览器是例外 (它有特殊行为)。

- 具体规则:

- 正常浏览器 (Chrome/Firefox等) :

- 不指定 `Domain` 时, Cookie 只能被设置时的精确域名访问。
 - 例如: `docs.foo.com` 设置的Cookie (无Domain属性), 不能被 `foo.com` 或 `api.foo.com` 使用。

- Internet Explorer (IE) :

- IE会自动将Cookie的Domain扩展到父域名 (如 `docs.foo.com` → `foo.com`), 这是与其他浏览器的差异。

2. "If Path is specified, the cookie is only used when requesting pages contained in the corresponding path."

- 含义:

如果设置了 `Path` 属性 (如 `Path=/admin`), 则只有请求的URL路径匹配该路径时, 浏览器才会在请求头中携带这个Cookie。

- 示例:

```
http
```

[复制](#)

```
Set-Cookie: auth=xyz; Path=/admin
```

- ☒ 会携带Cookie的请求:

`GET /admin/dashboard`、`GET /admin/users`

- ☐ 不会携带Cookie的请求:

`GET /home`、`GET /profile`

- 用途:

限制Cookie的作用范围, 提高安全性 (例如后台管理系统的Cookie不应在前台页面发送)。

3. "If Path is not specified, the path of the requested resource is used."

- 含义:

如果设置Cookie时不指定 `Path` 属性, 则浏览器会默认使用当前请求的路径作为Cookie的 `Path`。

- 示例:

1. 用户访问 `GET /products/details`, 服务器返回:

```
http
```

[复制](#)

```
Set-Cookie: cart_id=abc; # 未指定Path
```

2. 浏览器会自动将Path设为 `/products` (当前请求的父路径)。

3. 后续只有访问 `/products` 或其子路径 (如 `/products/list`) 时才会携带该Cookie。

Cookie Attributes (Cont.)

Cookies set by docs.foo.com

HTTP/1.0 200 OK

Set-Cookie: LSID=DQAAAK...Eaem_vYg; Path=/accounts; Expires=Wed, 13 Jan 2021

22:23:01 GMT; Secure; HttpOnly

Set-Cookie: HSID=AYQEVn...DKrdst; Domain=.foo.com; Path=/; Expires=Wed, 13 Jan 2021

22:23:01 GMT; HttpOnly

Set-Cookie: SSID=Ap4P...GTEq; Domain=.foo.com; Path=/; Expires=Wed, 13 Jan 2021

22:23:01 GMT; Secure; HttpOnly

Set-Cookie: made write conn=1295214458; Path=/; Domain=.foo.com

Set-Cookie: reg_fb_gate=deleted; Expires=Thu, 01 Jan 1970 00:00:01 GMT; Path=/;
Domain=.foo.com; HttpOnly

...

- The **Expires** and **Max-Age** attributes define the cookie's expiration.
 - **Expires** sets the time when the browser should delete the cookie.
 - If neither **Expires** nor **Max-Age** is set, the cookie is a *session cookie*.
- The **Secure** attribute instructs the browser to use the cookie only via secure/encrypted connections, e.g., HTTPS.
 - What if the Set-Cookie HTTP response is *not encrypted*?
- The **HttpOnly** attribute directs browsers to not expose the cookie through channels other than HTTP requests.
 - Prevents JavaScript code from *stealing* the cookie. Why?

Secure 属性的作用是：浏览器仅在 **HTTPS 加密连接** 中发送该Cookie，防止明文传输被窃取。

问题回答：

如果 **Set-Cookie** 的HTTP响应未加密（通过HTTP明文传输）：

1. **Cookie可能被中间人窃取**（如公共Wi-Fi嗅探）。
2. **但浏览器仍会遵守 **Secure** 规则：**
 - 后续请求中，浏览器**只有在HTTPS链接时才会携带该Cookie**，HTTP链接不会发送。
3. **矛盾点：**
 - 首次设置Cookie时已暴露（因为HTTP明文传输），但后续使用受保护。
 - **最佳实践：**确保 **Set-Cookie** 也通过HTTPS发送（全站HTTPS）。

一句话总结：

Secure 能防止Cookie在后续HTTP请求中泄露，但首次设置时仍需HTTPS保护，否则可能被截获。

1. 核心作用

HttpOnly 属性告诉浏览器：这个Cookie只能通过HTTP请求（如网页访问）自动传输，禁止JavaScript代码（如 `document.cookie`）读取或修改它。

2. 为什么需要它？

- **防止XSS攻击（跨站脚本攻击）**
恶意脚本如果注入网页，可能会尝试用 `document.cookie` 窃取用户的Cookie（如会话ID）。
如果Cookie标记了 **HttpOnly**，JavaScript无法读取，攻击者就无法直接盗用Cookie。

Cookie设置方式	JavaScript能否读取？	安全性
Set-Cookie: session=123	✅ 能读取	❌ 不安全（易被XSS窃取）
Set-Cookie: session=123; HttpOnly	❌ 不能读取	✅ 安全

- **不是万能的：**
HttpOnly 只能防JavaScript窃取，仍需配合其他措施（如 **Secure** 属性+HTTPS）防止网络嗅探。
- **现代Web必备：**
所有会话Cookie（如登录状态）都应启用 **HttpOnly**。

Client-side Access to Cookies

- Reading a cookie
 - `var cookies = document.cookie`
 - Only cookies in scope are returned
- Setting a new cookie
 - `document.cookie = "name=value; expires=...;"`
 - It does not overwrite all existing cookies
- Deleting a cookie
 - `document.cookie = "name=value; expires=PAST TIME;"`

1. 读取 Cookie

- 代码:

```
javascript
var cookies = document.cookie;
```

复制

- 作用:

获取当前页面所有可见的 Cookie (仅限当前域名和路径下未设置 HttpOnly 的 Cookie)。

- 返回值:

字符串格式, 如 "name1=value1; name2=value2"。

- 限制:

- 如果 Cookie 设置了 HttpOnly, JavaScript 无法读取 (防XSS攻击)。
- 只能读到符合当前页面 Domain 和 Path 范围的 Cookie。

2. 设置新 Cookie

- 代码:

```
javascript
document.cookie = "username=John; expires=Thu, 18 Dec 2025 12:00:00 UTC; path="/;
```

复制

- 特点:

- 不会覆盖其他 Cookie, 只会新增或修改指定的键值对。
- 需手动设置属性 (如 expires、path、domain), 否则使用默认值。

- 示例:

- 设置一个 30 天后过期的 Cookie:

```
javascript
var date = new Date();
date.setDate(date.getDate() + 30);
document.cookie = `theme=dark; expires=${date.toUTCString()}; path=/`;
```

复制

3. 删除 Cookie

- 原理:

通过设置 Cookie 的过期时间为过去的时间, 触发浏览器自动删除。

- 代码:

```
javascript
document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/";
```

复制

- 关键点:

- path 和 domain 必须与要删除的 Cookie 完全一致, 否则可能无效。
- 值可留空 (如 username=)。

Cookie的作用域

Cookie Scope

- Cookies are managed in a cookie jar per domain
- The browser automatically attaches the cookies in scope with the HTTP requests
- The Domain and Path attributes let the browser decide which websites to send a particular cookie
- The scope of cookie may not necessarily be the same as the URL of a website which sets it

1. Cookie 的存储机制

- "Cookie jar per domain" (按域名管理的Cookie罐子)

浏览器会为每个域名 (如 `example.com`) 单独存储一组Cookie, 就像为每个商店准备一个专属会员卡盒。

- 例如:
 - `taobao.com` 的Cookie存在一个“罐子”里。
 - `jd.com` 的Cookie存在另一个“罐子”里。
- 隔离性: A网站的JavaScript无法读取B网站的Cookie。

2. 自动携带Cookie的规则

- "浏览器自动附加作用域内的Cookie到HTTP请求"

当访问某个URL时, 浏览器会自动筛选符合以下条件的Cookie, 并添加到请求头中:

1. **域名匹配**: Cookie的 `Domain` 属性与当前URL域名一致 (或为其父域名)。
2. **路径匹配**: Cookie的 `Path` 属性是当前URL路径的父级 (或相同)。

示例:

- Cookie设置:

```
http
```

复制

```
Set-Cookie: cart_id=123; Domain=.example.com; Path=/shop
```

- 以下请求会携带该Cookie:

- ✓ `https://example.com/shop/checkout`
- ✓ `https://sub.example.com/shop`
- ✗ `https://example.com/home` (路径不匹配)
- ✗ `https://google.com` (域名不匹配)

3. Domain 和 Path 属性的作用

属性	功能	默认值
Domain	控制Cookie可被哪些域名访问	当前精确域名 (host-only)
Path	控制Cookie可被哪些路径访问	当前请求的父路径

- Domain 示例:

- `Domain=.example.com`: 允许 `example.com` 及其所有子域名 (如 `shop.example.com`) 使用该Cookie。
- 不设置 `Domain`: 仅限设置时的精确域名 (如 `docs.example.com` 的Cookie不能被 `example.com` 使用)。

- Path 示例:

- `Path=/admin`: 仅限 `/admin` 及其子路径 (如 `/admin/users`) 携带Cookie。
- 不设置 `Path`: 默认为当前请求的父路径 (如请求 `/products/1` 则 `Path=/products`)。



4. 特殊说明

- "Cookie的作用域不一定与设置它的网站URL相同"

- 服务器可以主动扩大Cookie的作用域 (例如: 从 `docs.example.com` 设置为 `Domain=.example.com`)。
- 用途: 实现跨子域名共享登录状态 (如 `user.example.com` 和 `shop.example.com` 共用同一会话Cookie)。

Cookie SOP

同源政策
Same-origin policy

- Recall that in SOP an origin is based on
 - scheme, hostname, port
- Cookies use a different definition of origins
- The cookie scope is based on
 - [scheme], domain, port

Cookie的“同源”判断放宽了部分限制，仅基于以下三要素：

- [Scheme] (可选协议) :
 - 若Cookie设置 **Secure** 属性，则仅限HTTPS使用。
 - 否则不校验协议 (HTTP/HTTPS可共享Cookie) 。
- Domain (域名) :
 - 显式设置 **Domain=.example.com** 时，子域名 (如 **shop.example.com**) 可共享。
 - 未设置时，仅限精确域名 (host-only) 。
- Port (端口) :
 - 通常忽略端口 (**example.com:80** 和 **example.com:8080** 默认共享Cookie) 。

Cookie Policy

- A website might try to set cookies for another website
- The browser checks if the website is allowed to set a cookie
- Valid Domain can be any domain suffix of the hostname in the URL, except for the top-level domain (TLD)
 - Example: a.abc.com
 - Allowed: .a.abc.com, .abc.com
 - Disallowed: .b.abc.com, .a.abcde.com, .com
- Path can be set to anything

Cookie Policy (Cont.)

如何自动决定携带哪些 cookie 发给服务器

- The browser decides which cookies to send when visiting a particular URL
- It sends all cookies in URL scope
 - The Domain of a cookie is a suffix of the domain in the URL
 - The Path of a cookie is a prefix of the path in the URL
 - Protocol = HTTPS is cookie has the secure attribute

1. 域名匹配规则 (Domain)

- 条件: Cookie的 **Domain** 必须是当前URL域名的后缀。
- 示例 (当前访问的URL为 `https://shop.example.com/products`) :

Cookie的 Domain 设置	是否携带	原因
<code>.example.com</code>	✓ 携带	是URL域名的父级后缀
<code>.shop.example.com</code>	✓ 携带	是URL域名的精确后缀
<code>example.com</code> (无点前缀)	✗ 不携带	需显式以 <code>.</code> 开头才允许子域名共享
<code>.other.com</code>	✗ 不携带	非当前域名的后缀

- 特殊限制:
浏览器禁止为顶级域名 (如 `.com`、`.org`) 设置Cookie, 否则所有子网站可共享同一Cookie (极端危险!)。

2. 路径匹配规则 (Path)

- 条件: Cookie的 **Path** 必须是当前URL路径的前缀 (即URL路径以Cookie的 **Path** 开头)。
- 示例 (当前访问的URL为 `https://example.com/admin/dashboard`) :

Cookie的 Path 设置	是否携带	原因
<code>/admin</code>	✓ 携带	URL路径 <code>/admin/dashboard</code> 以 <code>/admin</code> 开头
<code>/</code>	✓ 携带	所有路径均以 <code>/</code> 开头
<code>/dashboard</code>	✗ 不携带	URL路径不以 <code>/dashboard</code> 开头

- 默认行为:
若未设置 **Path**, 则默认为当前请求的父路径 (如请求 `/products/1` 则 `Path=/products`)。

3. 协议匹配规则 (Secure)

- 条件:
 - 若Cookie设置了 **Secure** 属性, 则仅当URL为HTTPS时才会携带。
 - 未设置 **Secure** 的Cookie, HTTP和HTTPS均可携带。

- 示例:

Cookie属性	访问 <code>http://example.com</code>	访问 <code>https://example.com</code>
<code>session=123; Secure</code>	✗ 不携带	✓ 携带
<code>session=123</code> (无Secure)	✓ 携带	✓ 携带

SOP: Cookies vs. DOM

- Cookies

默认情况下, 不同路径的Cookie是隔离的。
abc.com/x 无法直接接收 abc.com/y 的Cookie, 除非满足以下条件:

- Cookie 设置了 Path=/ (全站共享)。
- 属于同一域名 (Domain=abc.com)。

因为path不同

- abc.com/x does not receive cookies for abc.com/y

- DOM

因为二者同源,

- abc.com/x can access the DOM of abc.com/y

- Using JavaScript, abc.com/x is able to see the cookies of abc.com/y

- How?

- Exception?

• 方法:

- 如果Cookie 未设置 HttpOnly, 且 路径允许 (如 Path=/), 可通过 document.cookie 读取同域名的所有非 HttpOnly Cookie。
- 示例:

```
javascript 复制
// 在 abc.com/x 中读取当前域名的Cookie (需Path=/)
console.log(document.cookie); // 输出所有非HttpOnly的Cookie
```

• 例外情况:

- HttpOnly Cookie:** 禁止JavaScript读取 (即使路径匹配)。
- 路径限制:** 若Cookie的 **Path=/y**, 则 **abc.com/x** 无法读取。

Secure Cookies

- If an authentication cookie is set for bank.com
- Would the browser send the cookie to the following URLs?
 - `https://www.bank.com/`
 - `http://www.bank.com`
- Problem?
 - MITM attacks
- Fix
 - The Secure attribute

• Cookie设置:

http 复制

Set-Cookie: session=123; Domain=bank.com; Path=/ # 缺少 Secure!

• 浏览器行为:

访问的URL	是否发送Cookie?	风险
<code>https://www.bank.com/</code>	✓ 发送	安全 (HTTPS加密)
<code>http://www.bank.com</code>	✓ 发送	高危! Cookie被中间人窃取 (MITM)

• MITM攻击原理:

攻击者在公共Wi-Fi等网络拦截HTTP请求, 直接窃取Cookie, 冒充用户登录。

• 正确设置:

http 复制

Set-Cookie: session=123; Domain=bank.com; Path=/; Secure; HttpOnly

• 浏览器行为 (修复后):

访问的URL	是否发送Cookie?	原因
<code>https://www.bank.com/</code>	✓ 发送	HTTPS + Secure 匹配
<code>http://www.bank.com</code>	✗ 不发送	协议为HTTP, Secure Cookie被拦截

HttpOnly Cookies

- All included JavaScript execute in the context of the embedding frame/page
- A script loaded from xyz.com into bank.com can access the cookies of bank.com
- The cookies then can be leaked to xyz.com or arbitrary servers
 - How?
- Fix:
 - The HttpOnly attribute

Indirectly Bypassing SOP

- A related domain attacker can control the cookies on another domain
- Example: [a.example.com](#) and [b.example.com](#)
- The cookie jar may have a fixed storage limit
- [a.example.com](#) can try to set many cookies for [.example.com](#)
- When a user visits [b.example.com](#), the cookies set by the attacker are sent by the browser
- This indirectly bypasses SOP because [a.example.com](#) can influence [b.example.com](#)

- 场景设定：

- 主域名： `example.com`
- 子域名： `a.example.com`（攻击者控制）和 `b.example.com`（目标站点）。

- 攻击步骤：

1. 攻击者设置大量Cookie：

`a.example.com` 通过恶意脚本或页面，为父域名 `.example.com` 设置大量Cookie（达到浏览器存储上限）。

```
http
```

[复制](#)

```
Set-Cookie: junk1=xxx; Domain=.example.com; Path=/  
Set-Cookie: junk2=yyy; Domain=.example.com; Path=/  
...（重复直到填满Cookie存储）
```

2. 覆盖关键Cookie：

当用户访问 `b.example.com` 时，浏览器因存储空间不足，丢弃部分原有Cookie（如会话ID），导致 `b.example.com` 无法正常读取用户凭证。

3. 间接绕过SOP：

虽然 `a.example.com` 无法直接访问 `b.example.com` 的数据，但通过操纵共享的Cookie存储，影响了 `b.example.com` 的会话状态。

- Cookie的作用域规则：

- 子域名（如 `a.example.com`）可以为父域名（`.example.com`）设置Cookie，这些Cookie会被所有子域名（包括 `b.example.com`）共享。

- SOP的局限性：

SOP限制的是直接访问数据（如JS读取DOM），但通过资源耗尽攻击（如填满Cookie存储）可间接影响其他同源站点的行为。

Session Fixation

- The attack:
 - An attacker obtains a session ID from a web server
 - A victim is tricked into visiting the website under the session ID of the attacker
 - A victim completes web authentication using attacker's session ID
 - The attacker is now able to visit the website as the victim
- The initial step is not necessary if the application blindly accepts an existing session ID
- Session IDs should always be updated after login and never be “inherited”
- Cross-site scripting can help initialize the session ID

- **第一步：攻击者获取合法会话ID**

- 攻击者正常访问网站（如 `bank.com`），服务器分配一个会话ID（如 `sessionid=attacker123`）。
- 注：若网站允许用户自定义会话ID（如通过URL参数），此步骤可跳过。

- **第二步：诱骗受害者使用该会话ID**

- 通过钓鱼邮件/链接，诱导受害者点击已嵌入攻击者会话ID的URL。

复制

```
https://bank.com/login?sessionid=attacker123
```

- 受害者访问后，浏览器绑定此会话ID（但尚未登录）。

- **第三步：受害者完成登录**

- 受害者输入用户名密码登录，服务器将攻击者的会话ID与受害者账户关联。
- 此时 `sessionid=attacker123` 变为已认证状态。

- **第四步：攻击者劫持会话**

- 攻击者直接用原会话ID（`attacker123`）访问网站，无需密码即可操作受害者账户。

- **网站设计漏洞：**

- 登录前后会话ID不变（未重新生成）。
- 允许通过URL/表单参数直接设置会话ID（如 `?sessionid=xxx`）。

- **辅助手段：**

- 若网站存在XSS漏洞，攻击者可注入脚本强制设置会话ID：

```
javascript
```

复制

```
document.cookie = "sessionid=attacker123; path="/;
```

Cross-Site Request Forgery

Cross-Site Request Forgery

CSRF 跨站请求伪造

- In CSRF attacks, a malicious server or JavaScript code tricks the user (her/his browser) into performing actions on another web application
 - Posting a new message
 - Deleting a record
 - Initiating a transfer
- The attacker needs to trick the user into her/his web page at the right time

假设你登录了银行网站，然后又打开了一个恶意网站。恶意网站中嵌入了一段 JavaScript 代码，这段代码会向你的银行网站发送一个转账请求。由于你的浏览器已经登录了银行网站，银行会认为这个请求是合法的，从而完成转账操作。而你可能完全不知道这一切发生了。

CSRF攻击中，恶意网站无需知道你的Cookie具体值，而是利用浏览器“自动附加Cookie”的机制。

- 只要你是已登录状态（浏览器存有银行Cookie），任何向银行域名发起的请求（即使来自恶意网站）都会自动带上该Cookie。
- 恶意网站只需伪造请求格式（如转账API的URL和参数），无需窃取Cookie内容。

Cross-Site Request Forgery (Cont.)

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>Welcome</h1>
<p>Welcome to Evil's page!</p>




</body>
</html>
```



The browser automatically sends a request to this URL with the corresponding cookies!

CSRF Scenario



Client

1. Visits bank.com and establishes a session

2. 

5. Sends forged request w/ 

6. Bank acts on the request of the client

3. Visits evil.com

4. A page containing URL to bank is returned



Bank



Evil

CSRF Countermeasures

- Avoid using GET methods when exporting functionality
 - However, one can use JavaScript to create a form to send a POST request
- Referrer Validation
 - The Referer header field indicates which URL initiated the HTTP request
 - Referer: http://www.bank.com/transfer.html
 - This must be implemented at the **server** side
- Secret Token Validation
 - `<input type=hidden name="token" value="8x13..">`
 - This must be implemented at the **server** side

Referer Validation

- Referer may leak privacy-sensitive information
- Referer is not always available
 - Stripped by the browser for HTTPS -> HTTP transitions
 - Stripped according to user's preference
 - Stripped by local machine
 - Stripped by the organization
- Referer is none:
 - Default allow: less secure and more usable
 - Default deny: secure but less usable

1. Referer 是什么？

- 定义：

HTTP请求头中的 **Referer**（注意拼写错误，应为Referrer，但协议沿用历史拼写）字段，表示当前请求是从哪个页面链接发起的。

- 示例：

```
http
GET /transfer HTTP/1.1
Host: bank.com
Referer: https://bank.com/dashboard # 表示从dashboard页面跳转而来
```

2. Referer 的隐私泄露风险

- 敏感信息暴露：

- 若用户从含敏感信息的页面跳转（如 `https://hospital.com/patient/123`），**Referer** 可能泄露URL中的隐私数据。
 - 攻击者或中间人可通过拦截 **Referer** 获取用户行为轨迹。

- 实际案例：

用户从公司内网（`intranet.corp.com`）访问外部网站时，**Referer** 可能暴露内网结构。

3. Referer 可能被剥离的场景

场景	原因
HTTPS → HTTP 跳转	浏览器自动剥离HTTPS页面的Referer，防止安全降级（如从 <code>https://bank.com</code> 跳转到 <code>http://payment.com</code> ）。
用户隐私设置	浏览器选项（如Firefox的“严格跟踪保护”）或插件（如隐私罐）可能禁用Referer。
企业网络策略	公司防火墙或代理服务器可能过滤Referer头。
开发者主动移除	通过 <code><meta></code> 标签或CSP策略禁止发送Referer： <code><meta name="referrer" content="no-referrer"></code>

4. Referer 缺失时的两难选择

- 默认允许 (Default Allow)：

- 若 **Referer** 为空则放行请求。
 - 优点：用户体验好（不会因Referer缺失导致功能异常）。
 - 缺点：安全性低（攻击者可构造无Referer的CSRF请求）。

- 默认拒绝 (Default Deny)：

- 若 **Referer** 为空则拒绝请求。
 - 优点：安全性高。
 - 缺点：可能误杀合法请求（如隐私敏感用户无法使用功能）。

Secret Token Validation

- The server tags **each action** for **each user** with a **secret token**
- The token can be obtained **only if** the user has explicitly visited the **right page** for initiating that action
- The malicious web page would not contain the valid token
- The secret token needs to be difficult to predict
 - Not depending on publicly known information
 - Randomly generated

Credits

- Some slides are based on the slides of Giovanni Vigna, Raluca Ada Popa, and Zakir Durumeric